



ADMINISTRACIÓN DE SERVIDORES

DOCUMENTACIÓN SERVIDORES SSH

PROYECTO GRUPAL

José Luis Olmo Barberá
Victor Manuel Vidal Molina
Alvaro Griñón Martínez

CURSO 2024/25

Índice

1. Explicación contenido añadido a la infraestructura	2
1.1. Servidor SSH en Services	2
1.1.1. Requisitos:	2
1.1.2. Código del contenedor ssh_services en el docker-compose.yml: . . .	2
1.1.3. Código del dockerfile usado para crear una imagen personalizada: .	3
1.1.4. Código del script entrypoint.sh usado en el Dockerfile.	3
1.1.5. Especificaciones técnicas:	4
1.1.6. Explicación dockerfile:	4
1.1.7. Comprobaciones dentro del contenedor:	5
1.1.8. Acceso desde el host mediante ssh:	6
1.2. Servidor SSH en Development	7
1.2.1. Requisitos:	7
1.2.2. Código del contenedor ssh_development en el docker-compose.yml:	7
1.2.3. Código del dockerfile usado para crear una imagen personalizada: .	8
1.2.4. Código del script entrypoint.sh usado en el Dockerfile.	8
1.2.5. Especificaciones técnicas:	9
1.2.6. Explicación dockerfile:	9
1.2.7. Comprobaciones dentro del contenedor:	10
1.2.8. Acceso desde el host mediante ssh:	11

1. Explicación contenido añadido a la infraestructura

Durante esta fase de implementación, se han incorporado dos nuevos servidores SSH a la infraestructura anterior, cada uno con una configuración específica de autenticación según su red correspondiente. A continuación se detallan las características y configuraciones implementadas en cada uno:

1.1. Servidor SSH en Services

1.1.1 Requisitos:

Se requiere implementar un servidor SSH en la red Services al que los usuarios puedan acceder con el par de claves público/privado, sin necesidad de hacer login con usuario/clave.

1.1.2 Código del contenedor `ssh_services` en el `docker-compose.yml`:

```
# Bloque Servidores SSH
ssh_services:
  build: ./services/ssh_services
  container_name: ssh_services
  restart: always
  cap_add:
    - NET_ADMIN
  dns:
    - 172.20.0.6
  networks:
    bridge_local: {}
    services_net:
      ipv4_address: 172.20.0.9
  ports:
    - "2222:22"
  volumes:
    - ./services/ssh_services/authorized_keys:/home/john/.ssh/authorized_keys:ro
```

Figura 1: Contenedor `ssh_services`.

1.1.3 Código del dockerfile usado para crear una imagen personalizada:

```
# ssh_services/Dockerfile
FROM debian:bullseye

RUN apt update && apt install -y openssh-server sudo iputils-ping dnsutils iproute2
RUN useradd -m john && echo 'john:john' | chpasswd && usermod -aG sudo john && \
    mkdir -p /home/john/.ssh && chmod 700 /home/john/.ssh

COPY authorized_keys /home/john/.ssh/authorized_keys

RUN chown -R john:john /home/john/.ssh && chmod 600 /home/john/.ssh/authorized_keys

RUN mkdir /var/run/sshd

# Deshabilita el login por contraseña
RUN sed -i 's/^#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config

COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

EXPOSE 22

ENTRYPOINT ["/entrypoint.sh"]
CMD ["/usr/sbin/sshd", "-D"]
```

Figura 2: Dockerfile para crear imagen personalizada de ssh_services.

1.1.4 Código del script entrypoint.sh usado en el Dockerfile.

```
#!/bin/bash

ip route del default 2>/dev/null

iface_srv=$(ip -o -4 addr show | grep '172\.20\.' | awk '{print $2}')

if [[ -n "$iface_srv" ]]; then
    echo "Asignando gateway 172.20.0.1 a traves de $iface_srv"
    ip route add default via 172.20.0.1 dev "$iface_srv"
fi

echo "nameserver 172.20.0.6" > /etc/resolv.conf

exec "$@"
```

Figura 3: Contenido script ejecutado en el dockerfile de ssh_services.

1.1.5 Especificaciones técnicas:

- Nombre del contenedor: `ssh_services`.
- IP asignada: `172.20.0.9`.
- DNS: `172.20.0.6`.
- Puerto de acceso: `2222`.
- Redes: `services_net` que lo conecta a la infraestructura y `bridge_local` permite conectar el host con la infraestructura `macvlan`.
- Objetivo de seguridad: Autenticación mediante clave pública/privada, sin contraseña.

1.1.6 Explicación dockerfile:

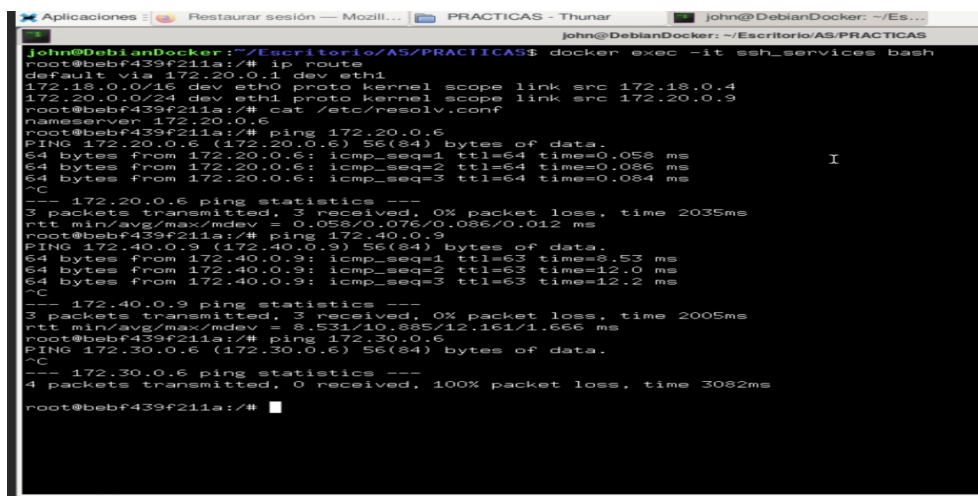
- Actualizamos e instalamos los paquetes necesarios en el contenedor.
- Creamos el usuario John y damos permisos, que aunque no se especificaba nada, al igual que en la práctica anterior lo hemos añadido a todos los contenedores.
- Copiamos el archivo `authorized_keys` desde el host al contenedor y damos permisos, previamente deberemos generar dicho archivo ya que necesario para poder acceder mediante el par de claves pública/privada, para ello debemos acceder desde nuestro host a la carpeta `ssh_services` desde la terminal y ejecutar el siguiente comando:

```
ssh-keygen -t rsa -b 4096 -C "joseluis.olmobarbera@alum.uca.es"
```

- Creamos el directorio `/var/run/sshd`.
- Deshabilitamos el login por contraseña.
- Exponemos el puerto `22`.
- Copiamos el archivo `entrypoint.sh` desde el host al contenedor, damos permisos y posteriormente lo ejecutamos, con el código contenido en el script `entrypoint.sh` nos aseguramos que el contenedor usa el gateway y DNS correctos.

1.1.7 Comprobaciones dentro del contenedor:

Accedemos al contenedor y hacemos una serie de comprobaciones como asegurarnos que el gateway y el dns son los correctos y probamos a hacer ping al dns, que es un contenedor de la propia red services y también a un contenedor de la red development, por ejemplo el otro servidor ssh, vemos que ambos pings llegan correctamente, sin embargo el último ping, con destino a una IP de la red production no llega, esto es lo correcto según el guión de la práctica anterior, todo lo descrito se ve en la siguiente imagen:



```
John@DebianDocker: ~/Escritorio/AS/PRACTICAS$ docker exec -it ssh_services bash
root@beb439f211a:/# ip route
default via 172.20.0.1 dev eth1
172.18.0.0/16 dev eth0 proto kernel scope link src 172.18.0.4
172.20.0.0/24 dev eth1 proto kernel scope link src 172.20.0.9
root@beb439f211a:/# cat /etc/resolv.conf
nameserver 172.20.0.6
root@beb439f211a:/# ping 172.20.0.6
PING 172.20.0.6 (172.20.0.6) 56(84) bytes of data.
64 bytes from 172.20.0.6: icmp_seq=1 ttl=64 time=0.058 ms
64 bytes from 172.20.0.6: icmp_seq=2 ttl=64 time=0.086 ms
64 bytes from 172.20.0.6: icmp_seq=3 ttl=64 time=0.084 ms
^C
--- 172.20.0.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2035ms
rtt min/avg/max/mdev = 0.058/0.076/0.086/0.012 ms
root@beb439f211a:/# ping 172.40.0.9
PING 172.40.0.9 (172.40.0.9) 56(84) bytes of data.
64 bytes from 172.40.0.9: icmp_seq=1 ttl=63 time=8.53 ms
64 bytes from 172.40.0.9: icmp_seq=2 ttl=63 time=12.0 ms
64 bytes from 172.40.0.9: icmp_seq=3 ttl=63 time=12.2 ms
^C
--- 172.40.0.9 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 8.531/10.885/12.161/1.666 ms
root@beb439f211a:/# ping 172.30.0.6
PING 172.30.0.6 (172.30.0.6) 56(84) bytes of data.
^C
--- 172.30.0.6 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3082ms
root@beb439f211a:/#
```

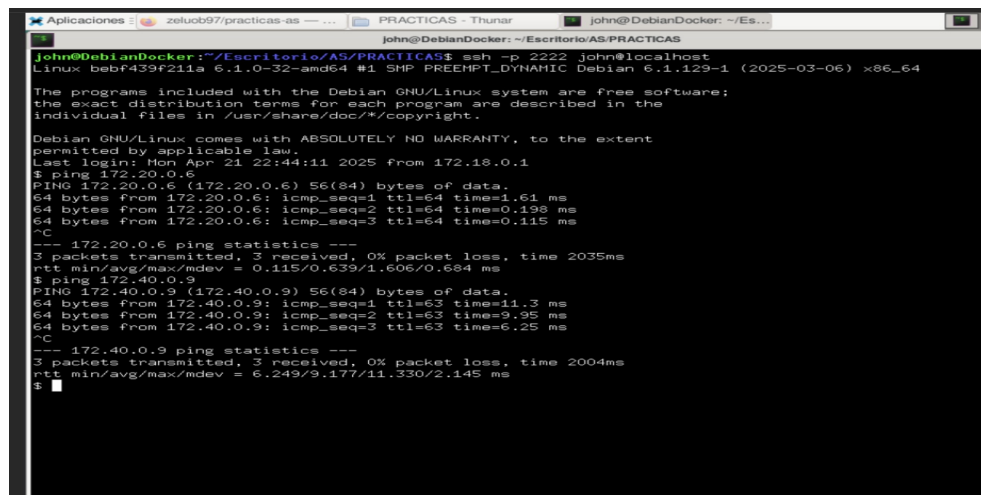
Figura 4: Comprobaciones dentro del contenedor ssh_services.

1.1.8 Acceso desde el host mediante ssh:

Para acceder, primero debemos levantar el docker-compose.yml y desde la terminal de nuestro host, fuera de cualquier contenedor, ejecutamos el siguiente comando:

```
ssh -p 2222 john@localhost
```

Podemos ver el resultado en la imagen:



```
john@DebianDocker: ~/Escritorio/AS/PRACTICAS$ ssh -p 2222 john@localhost
Linux bebf439f211a 6.1.0-32-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.129-1 (2025-03-06) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Apr 21 22:44:11 2025 from 172.18.0.1
$ ping 172.20.0.6
PING 172.20.0.6 (172.20.0.6) 56(84) bytes of data:
64 bytes from 172.20.0.6: icmp_seq=1 ttl=64 time=1.61 ms
64 bytes from 172.20.0.6: icmp_seq=2 ttl=64 time=0.138 ms
64 bytes from 172.20.0.6: icmp_seq=3 ttl=64 time=0.115 ms
^C
--- 172.20.0.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2035ms
rtt min/avg/max/mdev = 0.115/0.639/1.606/0.684 ms
$ ping 172.40.0.9
PING 172.40.0.9 (172.40.0.9) 56(84) bytes of data:
64 bytes from 172.40.0.9: icmp_seq=1 ttl=63 time=11.3 ms
64 bytes from 172.40.0.9: icmp_seq=2 ttl=63 time=9.95 ms
64 bytes from 172.40.0.9: icmp_seq=3 ttl=63 time=6.25 ms
^C
--- 172.40.0.9 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 6.249/9.177/11.330/2.145 ms
$
```

Figura 5: Conexión por ssh desde el host al contenedor ssh.services.

En la imagen como podemos observar hemos realizado la conexión por ssh al contenedor y hemos hecho varios pings para asegurarnos su funcionamiento como a la ip del dns y otro ping al servidor SSH de la red development, todo funciona correctamente.

1.2. Servidor SSH en Development

1.2.1 Requisitos:

Se requiere implementar un servidor SSH en la red Development al que los usuarios puedan acceder con su usuario y contraseña, además se requiere implementar Google Authenticator como método extra de seguridad.

1.2.2 Código del contenedor ssh_development en el docker-compose.yml:

```
ssh_development:
  build: ./development/ssh_development
  container_name: ssh_development
  restart: always
  cap_add:
    - NET_ADMIN
  dns:
    - 172.20.0.6
  networks:
    bridge_local: {}
    development_net:
      ipv4_address: 172.40.0.9
  ports:
    - "2223:22"
  volumes:
    - sshdev_data:/home/john
```

Figura 6: Contenedor ssh_development.

1.2.3 Código del dockerfile usado para crear una imagen personalizada:

```
# ssh_development/Dockerfile
FROM debian:bullseye

RUN apt update && apt install -y openssh-server libpam-google-authenticator sudo iputils-ping dnsutils iproute2

RUN useradd -m john && echo 'john:john' | chpasswd && usermod -aG sudo john
RUN mkdir /var/run/sshd

# Configurar SSH para usar Google Authenticator y contraseña
RUN sed -i 's/^#*ChallengeResponseAuthentication.*/ChallengeResponseAuthentication yes/' /etc/ssh/sshd_config && \
    sed -i 's/^#*PasswordAuthentication.*/PasswordAuthentication yes/' /etc/ssh/sshd_config && \
    sed -i 's/^#*UsePAM.*/UsePAM yes/' /etc/ssh/sshd_config

# Configurar PAM para Google Authenticator
RUN echo "auth required pam_google_authenticator.so" >> /etc/pam.d/sshd

COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

EXPOSE 22

ENTRYPOINT ["/entrypoint.sh"]
CMD ["/usr/sbin/sshd", "-D"]
```

Figura 7: Dockerfile para crear imagen personalizada de ssh_development.

1.2.4 Código del script entrypoint.sh usado en el Dockerfile.

```
#!/bin/bash

ip route del default 2>/dev/null

iface_dev=$(ip -o -4 addr show | grep '172\.40\..' | awk '{print $2}')

if [[ -n "$iface_dev" ]]; then
    echo "Asignando gateway 172.40.0.1 a traves de $iface_dev"
    ip route add default via 172.40.0.1 dev "$iface_dev"
fi

echo "nameserver 172.20.0.6" > /etc/resolv.conf

exec "$@"
```

Figura 8: Contenido script ejecutado en el dockerfile de ssh_development.

1.2.5 Especificaciones técnicas:

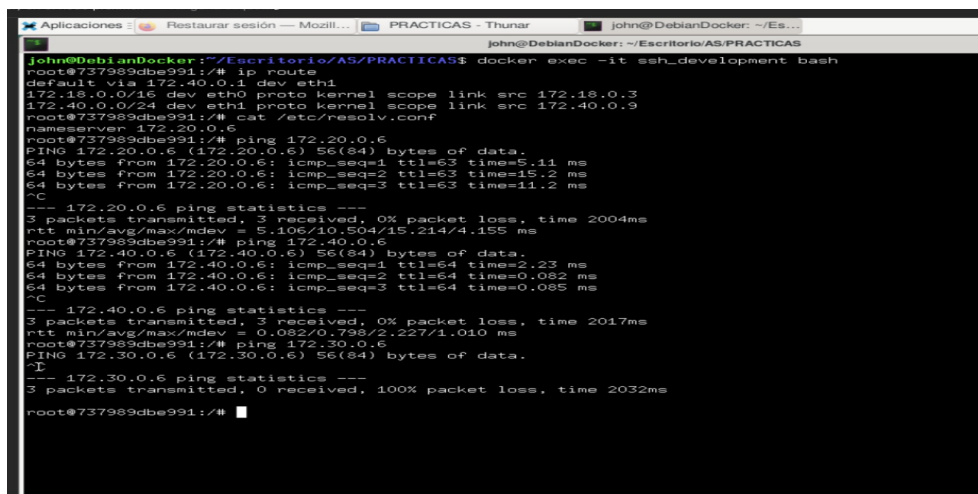
- Nombre del contenedor: `ssh_development`.
- IP asignada: `172.40.0.9`.
- DNS: `172.20.0.6`.
- Puerto de acceso: `2223`.
- Redes: `services_net` que lo conecta a la infraestructura y `bridge_local` permite conectar el host con la infraestructura `macvlan`.
- Objetivo de seguridad: Autenticación mediante usuario y contraseña + Google Authenticator.

1.2.6 Explicación `dockerfile`:

- Actualizamos e instalamos los paquetes necesarios en el contenedor.
- Creamos el usuario `John` y damos permisos, que aunque no se especificaba nada, al igual que en la práctica anterior lo hemos añadido a todos los contenedores.
- Creamos el directorio `/var/run/sshd`.
- Configuramos SSH para acceder mediante usuario y contraseña + Google Authenticator.
- Configuramos PAM para Google Authenticator.
- Exponemos el puerto `22`.
- Copiamos el archivo `entrypoint.sh` desde el host al contenedor, damos permisos y posteriormente lo ejecutamos, con el código contenido en el script `entrypoint.sh` nos aseguramos que el contenedor usa el gateway y DNS correctos.

1.2.7 Comprobaciones dentro del contenedor:

Accedemos al contenedor y hacemos una serie de comprobaciones como asegurarnos que el gateway y el dns son los correctos y probamos a hacer ping al dns, que es un contenedor de la red services y también a un contenedor de la red development, por ejemplo al contenedor mysql, vemos que ambos pings llegan correctamente, sin embargo el último ping, con destino a una IP de la red production no llega, esto es lo correcto según el guión de la práctica anterior, todo lo descrito se ve en la siguiente imagen:



```
john@DebianDocker: ~/Escritorio/AS/PRACTICAS$ docker exec -it ssh_development bash
root@737989dbe991:/# ip route
default via 172.40.0.1 dev eth1
172.18.0.0/16 dev eth0 proto kernel scope link src 172.18.0.3
172.40.0.0/24 dev eth1 proto kernel scope link src 172.40.0.9
root@737989dbe991:/# cat /etc/resolv.conf
nameserver 172.20.0.6
root@737989dbe991:/# ping 172.20.0.6
PING 172.20.0.6 (172.20.0.6) 56(84) bytes of data:
64 bytes from 172.20.0.6: icmp_seq=1 ttl=63 time=5.11 ms
64 bytes from 172.20.0.6: icmp_seq=2 ttl=63 time=15.2 ms
64 bytes from 172.20.0.6: icmp_seq=3 ttl=63 time=11.2 ms
^C
--- 172.20.0.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 5.106/10.504/15.214/4.155 ms
root@737989dbe991:/# ping 172.40.0.6
PING 172.40.0.6 (172.40.0.6) 56(84) bytes of data:
64 bytes from 172.40.0.6: icmp_seq=1 ttl=64 time=2.23 ms
64 bytes from 172.40.0.6: icmp_seq=2 ttl=64 time=0.082 ms
64 bytes from 172.40.0.6: icmp_seq=3 ttl=64 time=0.085 ms
^C
--- 172.40.0.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2017ms
rtt min/avg/max/mdev = 0.082/0.798/2.227/1.010 ms
root@737989dbe991:/# ping 172.30.0.6
PING 172.30.0.6 (172.30.0.6) 56(84) bytes of data:
--- 172.30.0.6 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2032ms
^C
root@737989dbe991:/#
```

Figura 9: Comprobaciones dentro del contenedor ssh_development.

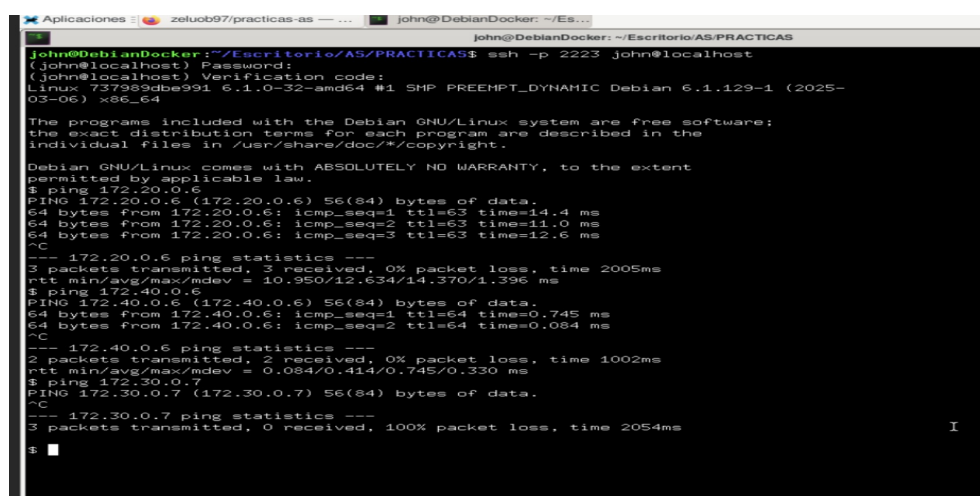
1.2.8 Acceso desde el host mediante ssh:

Para acceder, primero debemos asegurarnos que el docker-compose.yml esta up y desde la terminal de nuestro host, fuera de cualquier contenedor, ejecutamos el siguiente comando:

```
ssh -p 2223 john@localhost
```

A continuación, debemos introducir nuestro usuario y contraseña, seguido de el código generado por la aplicación Google Authenticator, si es la primera vez que iniciamos sesión, nos dará un código QR o una llave, ese código o llave se introduce en Google Authenticator y esta genera un número aleatorio de 6 dígitos, que es el que usaremos para hacer login.

Podemos ver el resultado en la imagen:



```
john@DebianDocker: ~/Escritorio/AS/PRACTICAS
john@DebianDocker: ~/Escritorio/AS/PRACTICAS$ ssh -p 2223 john@localhost
(john@localhost) Password:
(john@localhost) Verification code:
Linux 737989db991 6.1.0-32-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.129-1 (2025-03-06) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
$ ping 172.20.0.6
PING 172.20.0.6 (172.20.0.6) 56(84) bytes of data:
64 bytes from 172.20.0.6: icmp_seq=1 ttl=63 time=14.4 ms
64 bytes from 172.20.0.6: icmp_seq=2 ttl=63 time=11.0 ms
64 bytes from 172.20.0.6: icmp_seq=3 ttl=63 time=12.6 ms
^C
--- 172.20.0.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 10.950/12.634/14.370/1.396 ms
$ ping 172.40.0.6
PING 172.40.0.6 (172.40.0.6) 56(84) bytes of data:
64 bytes from 172.40.0.6: icmp_seq=1 ttl=64 time=0.745 ms
64 bytes from 172.40.0.6: icmp_seq=2 ttl=64 time=0.084 ms
^C
--- 172.40.0.6 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.084/0.414/0.745/0.330 ms
$ ping 172.30.0.7
PING 172.30.0.7 (172.30.0.7) 56(84) bytes of data:
^C
--- 172.30.0.7 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2054ms
$
```

Figura 10: Conexión por ssh desde el host al contenedor ssh_development.

Al igual que en el otro servidor SSH, vamos a realizar varios pings, uno al DNS, en la red services, otro al contenedor mysql ubicado en la propia red development, los cuales llegan a su destino sin problemas, por último hacemos ping a un contenedor de la red production, como vemos, este no llega a su destino, lo cual es correcto debido al aislamiento de esta red implementado en la práctica anterior.